1.2.57

Rushil Shanmukha Srinivas
EE25BTECH11057
Electrical Enggineering ,
IIT Hyderabad.

March 25, 2026

1. Problem
2. Solution
   - Using Lagrange Interpolation
   - Plots
3. Python Code

Problem Statement

Question : The graph y = f(x) passes through (0,-3) , (1,-1) , (2,3) . Using Lagrange interpolation, the x-value where y = 0 is :

1. 1.375
2. 1.475
3. 1.575
4. 1.675

Lagrange Interpolation

Given Points are

$$(y_0, x_0) = (-3, 0) \tag{3.1}$$
$$(y_1, x_1) = (-1, 1) \tag{3.2}$$
$$(y_2, x_2) = (3, 2) \tag{3.3}$$

The Lagrange formula for x as a function of y is given by :

$$x(y) = \sum_{i=0}^{n} x_i L_i(y) \tag{3.4}$$

where $L_i(y)$ are the basis polynomials.

For y = 0 :

$$L_0(y) = \frac{(y - y_1)(y - y_2)}{(y_0 - y_1)(y_0 - y_2)} \tag{3.5}$$

$$L_0(0) = \frac{(0 - (-1))(0 - 3)}{(-3 - (-1))(-3 - 3)} \tag{3.6}$$

$$L_0(0) = \frac{1(-3)}{(-2)(-6)} \tag{3.7}$$

$$L_0(0) = \frac{-3}{12} = -0.25 \tag{3.8}$$

$$L_1(y) = \frac{(y - y_0)(y - y_2)}{(y_1 - y_0)(y_1 - y_2)} \tag{3.9}$$

$$L_1(0) = \frac{(0 - (-3))(0 - 3)}{(-1 - (-3))(-1 - 3)} \tag{3.10}$$

$$L_1(0) = \frac{3(-3)}{(2)(-4)} \tag{3.11}$$

$$L_1(0) = \frac{-9}{-8} = 1.125 \tag{3.12}$$

$$L_2(y) = \frac{(y - y_0)(y - y_1)}{(y_2 - y_0)(y_2 - y_1)} \tag{3.13}$$

$$L_2(0) = \frac{(0 - (-3))(0 - (-1))}{(3 - (-3))(3 - (-1))} \tag{3.14}$$

$$L_2(0) = \frac{3(1)}{(6)(4)} \tag{3.15}$$

$$L_2(0) = \frac{3}{24} = 0.125 \tag{3.16}$$

Calculation of Final X value

Multiply each x_i value by its corresponding basis polynomial and sum them

$$x(0) = (0)(-0.25) + (1)(1.125) + (2)(0.125) \tag{3.17}$$

$$x(0) = 0 + 1.125 + 0.250 = 1.375 \tag{3.18}$$

The x value where y = 0 is 1.375

Plots

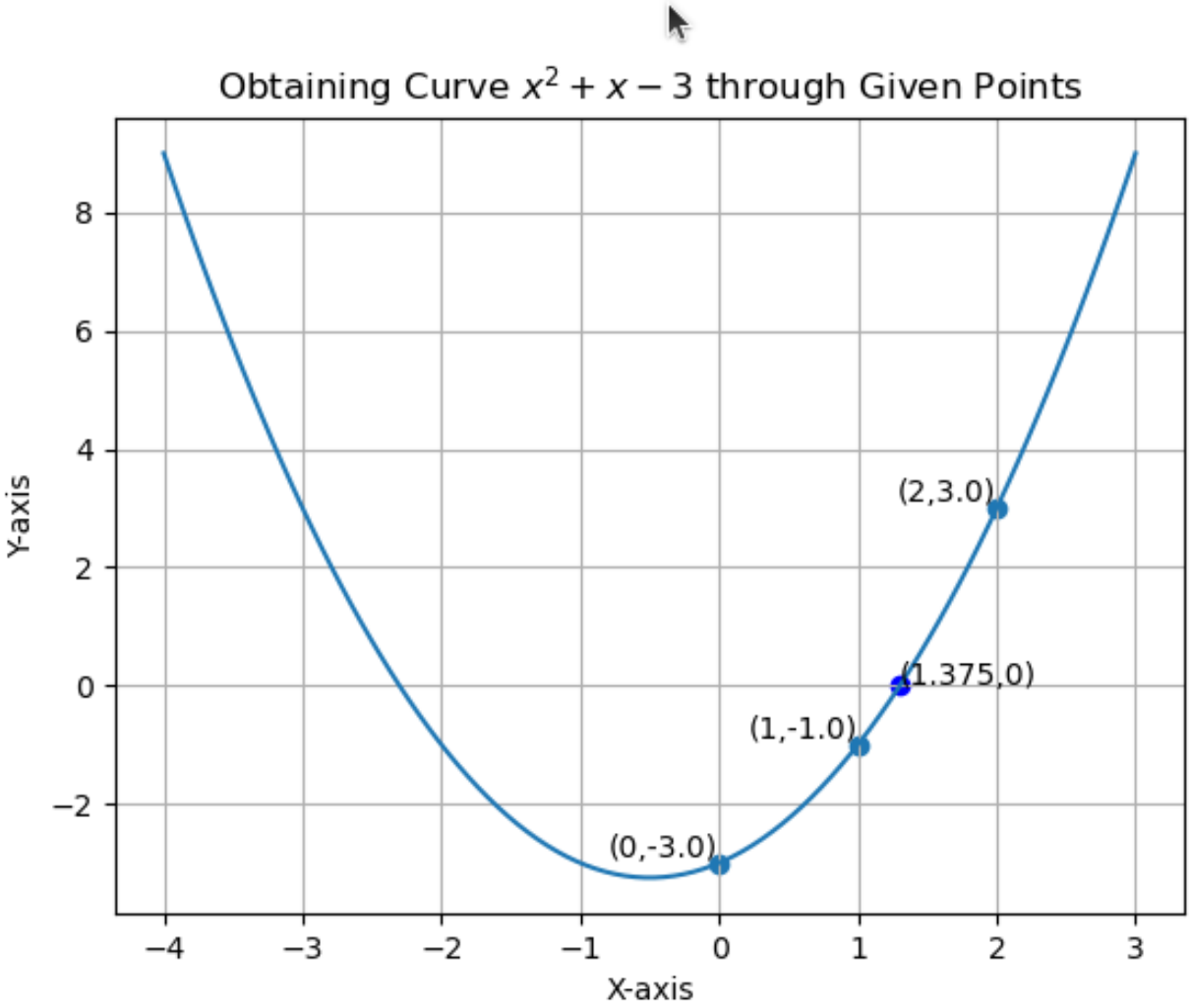

Figure: Parabola passing through given points

Python Code for Plotting

```
import numpy as np
import matplotlib.pyplot as plt

# Given points
x_points = np.array([0, 1, 2])
y_points = np.array([-3, -1, 3])

# Find polynomial coefficients (degree 2)
coeffs = np.polyfit(x_points, y_points, 2)

# Create polynomial function
poly = np.poly1d(coeffs)

# Generate x values for smooth curve
x_vals = np.linspace(-4, 3, 100)
y_vals = poly(x_vals)

# Plot curve
plt.plot(x_vals, y_vals)
```

```python
# Plot given points
plt.scatter(x_points, y_points)

label_x = [0, 1, 2] # points where you want labels
label_y = poly(label_x)

for x, y in zip(label_x, label_y):
    plt.text(x, y, f'({x},{round(y,2)})', fontsize=10,
             ha='right', va='bottom')
   # Add blue dot (same as other points)
plt.scatter(1.3, 0, color='blue')

plt.text(1.3, 0, '(1.375,0)', fontsize=10)

# Labels
plt.xlabel('X-axis')
plt.ylabel('Y-axis')
plt.title('Obtaining Curve $x^2+x-3$ through Given Points')

plt.grid()
plt.show()
```

Python Code for verification

```python
import numpy as np

# Given points
x_points = np.array([0, 1, 2])
y_points = np.array([-3, -1, 3])

# Find polynomial (degree 2)
coeffs = np.polyfit(x_points, y_points, 2)
poly = np.poly1d(coeffs)

# Print polynomial
print("Polynomial:")
print(poly)

# Verify each point
print("\nVerification:")
for x, y in zip(x_points, y_points):
    y_calc = poly(x)
    print(f"x = {x} -> Calculated y = {y_calc}, Actual y = {y}")
```

```python
# Find values of x when y = 0
roots = np.roots(coeffs)

print("\nValues of x when y = 0:")
for r in roots:
    print(r)
```